

Forking Codex to talk to any endpoint

Emil Lindfors | August 11, 2026

This is the technical companion to [Which LLM are we allowed to use?](#). That post argues why a consultancy needs its coding agent decoupled from any one model vendor. This one shows the code.

Short version: we forked [OpenAI's Codex CLI](#), restored the Chat Completions wire API that upstream removed, and ended up with an agent that talks to a local Ollama server, OpenRouter, Azure OpenAI, DeepInfra, or any other OpenAI-compatible endpoint by changing one line of TOML.

The problem

Upstream Codex used to support two wire protocols: OpenAI's newer **Responses** API and the older **Chat Completions** API that essentially the entire ecosystem implements. In [discussion #7782](#) they dropped Chat Completions and went Responses-only.

That is a defensible decision for a first-party tool. Responses is where the interesting features live: server-side reasoning items, encrypted reasoning, remote compaction, WebSocket transport. But it has one consequence, which is that **Codex can only speak to OpenAI-shaped endpoints**.

Everything else exposes Chat Completions and nothing else. OpenRouter, DeepInfra, Groq, Together, Ollama, LM Studio, vLLM, Scaleway's serverless inference, most EU sovereign providers. The usual workaround is a translation proxy: run [LiteLLM](#) locally, have it expose a `/v1/responses` endpoint, and let it translate down to Chat Completions.

That works. We ran it that way for a while. But a proxy is a process to start, a port to keep free, a config file to keep in sync, and a thing to explain to every colleague you onboard. For a tool meant to be handed to consultants who just want to get work done, it is a dealbreaker.

So we put `wire_api = "chat"` back.

Quickstart

If you just want a working agent against a cheap model, this is the whole thing. On Windows:

```
irm https://raw.githubusercontent.com/Fraktal-Norge-AS/fraktal-codex/fraktal/main/or-setup.ps1 -OutFile or-setup.ps1
.\or-setup.ps1
```

The script asks for an **OpenRouter key** and does the rest: downloads the binary, verifies its SHA-256 against the published checksum, puts it on `PATH`, stores the key as a user environment variable, and writes the config. Then open a **new** terminal, since `PATH` is read at process start, and:

```
cd C:\path\to\project
fraktal                                # interactive TUI
fraktal exec "what does this repo do?" # one-shot, non-interactive
```

Useful flags:

```
.\or-setup.ps1 -DryRun                # print every action, change
nothing
.\or-setup.ps1 -Model "deepseek/deepseek-v4-pro" # a different model
.\or-setup.ps1 -ApiKey "sk-or- ..."          # non-interactive
```

Three things the script does that are worth stealing if you write your own installer.

It never overwrites an existing config. If `~/.fraktal/config.toml` already exists, pointing at a local model say, it backs the file up, appends only the missing provider block, and writes the model selection into a separate *profile overlay* instead of hijacking your default. You opt in per run with `fraktal --profile openrouter`. Re-running it is safe.

It warns about `PATH` shadowing. We append to `PATH`, so an earlier entry (a `cargo install`ed build, typically) keeps winning and the binary you just downloaded never runs. Silent shadowing is worse than a warning:

```
$resolved = Get-Command fraktal -ErrorAction SilentlyContinue
if ($resolved -and $resolved.Source -and
    ($resolved.Source.TrimEnd('\') -ne $exePath.TrimEnd('\'))) {
    Write-Warn "Another fraktal is earlier on PATH and will be used instead:"
    Write-Warn "  $($resolved.Source)"
}
```

It reads the checksum from a file rather than from `.Content`. This one cost me twenty minutes. For a response PowerShell doesn't consider text, `Invoke-WebRequest`'s `.Content` is a `Byte[]`, and `-split` over a `Byte[]` yields the first byte, `53`, ASCII `'5'`, rather than the hash. You get a checksum mismatch on a download that was completely fine. Write it to a file and `Get-Content -Raw` it.

The config file

Configuration lives in `~/.fraktal/config.toml` (`FRAKTAL_HOME` overrides the path). Here is the shape, with all four providers we actually use. Comments translated to English; the repo's are in Norwegian.

```
# Default: our own Ollama server (requires VPN / office network)
model_provider = "fraktal-ollama"
model = "qwen3.5:27b"

[model_providers.fraktal-ollama]
name = "Fraktal Ollama (ml-dev-titan)"
base_url = "http://ml-dev-titan.example.internal:11434/v1"
discover_models = true # [fork] list what the server actually has in /model

# OpenRouter, direct – no translation proxy
[model_providers.fraktal-openrouter]
name = "OpenRouter"
base_url = "https://openrouter.ai/api/v1"
wire_api = "chat" # [fork] direct Chat Completions
requires_openai_auth = false
env_key = "OPENROUTER_API_KEY"
discover_models = true

# Azure OpenAI, with Entra ID auth via an external helper
[model_providers.fraktal-azure]
name = "Fraktal Azure OpenAI"
base_url = "https://example-aoai.openai.azure.com/openai/deployments/gpt-5-codex"
wire_api = "responses"
requires_openai_auth = false

[model_providers.fraktal-azure.http_headers]
"api-version" = "2026-02-01"

[model_providers.fraktal-azure.auth]
command = "fraktal-auth"
args = ["token", "--scope", "https://cognitiveservices.azure.com/.default"]
refresh_interval_ms = 300000
timeout_ms = 10000
```

Three things to note.

Profiles are separate files, not TOML tables. A profile `x` is an overlay file `~/.fraktal/x.config.toml` layered on top of `config.toml` when you run `fraktal --profile x`. It inherits `[model_providers.*]`, `[features]` and `[mcp_servers]` from the base config, so an overlay is usually two lines:

```
# ~/.fraktal/openrouter.config.toml
model_provider = "fraktal-openrouter"
model = "deepseek/deepseek-v4-flash-0731"
```

Use the dated model slug. `deepseek/deepseek-v4-flash` resolves to the older `0423` snapshot. The `-0731` slug is both newer *and* cheaper, at 0.09/0.18 per million tokens against 0.14/0.28. Check the exact slug and current price on the provider's model page before switching; this is the kind of thing that changes under you.

The Azure auth block is the interesting one. `command` is an external process the agent shells out to every `refresh_interval_ms` to get a bearer token on stdout. That gives you corporate SSO (device-code login against Entra ID, refresh token in the OS keyring, per-user audit trail) without the agent knowing anything about Entra. Any credential source that can print a token to stdout drops in the same way.

The patches that matter

The fork is about twenty commits on top of upstream, one commit per intent, all prefixed `[fraktal]` so the series stays droppable and rebasable. Most are cosmetic: binary rename, telemetry off, update check repointed, CI gated. Three of them do the real work.

1. Restoring `wire_api = "chat"`

This is the one that makes everything else possible. About 1,400 lines across fourteen files:

```
codex-rs/codex-api/src/endpoint/chat.rs      | 98 ++
codex-rs/codex-api/src/requests/chat.rs      | 496 ++
codex-rs/codex-api/src/sse/chat.rs           | 582 ++
codex-rs/core/src/client.rs                  | 123 ++
codex-rs/model-provider-info/src/lib.rs      | 22 +-
codex-rs/tools/src/tool_spec.rs              | 38 ++
...
```

The work breaks down as:

- `WireApi::Chat` back in `codex-model-provider-info`, including deserialisation of `wire_api = "chat"` so the TOML above parses.
- `ChatRequestBuilder`, the chat SSE parser, and `ChatClient` reinstated in `codex-api`, adapted to the current `EndpointSession` architecture rather than pasted back verbatim, since the transport layer had been reorganised since the removal.
- `create_tools_json_for_chat_completions_api` back in `codex-tools`. Chat Completions and Responses describe tools differently; the agent needs both encoders.

- A `WireApi::Chat` arm in core's client (`stream_chat_completions`) mirroring the Responses path.

Two lessons from doing this as a *restoration* rather than a fresh implementation.

Don't resurrect the old code; port the old behaviour. The temptation is to `git revert` the removal commit. It won't compile, and if you force it into compiling you get an implementation wired to an architecture that no longer exists. The transport now comes from `build_api_transport`, errors go through `provider.map_api_error`, and `map_response_stream / handle_unauthorized` take the provider. Making the chat path mirror the Responses path structurally is what keeps the patch rebasable, because when upstream refactors the transport again, both arms move together.

Some upstream types can't cross the protocol boundary at all. During a rebase, upstream added `InputAudio` to `ContentItem` and `FunctionCallOutputContentItem`. Chat Completions has no representation for it. We drop it as encrypted content on the chat path and document that, rather than invent a lossy mapping that silently corrupts a conversation.

There is a real trade-off here and it should be stated plainly: **the chat path has no Responses-native features**. No server-side reasoning items, no encrypted reasoning, no remote compaction, no `output_schema`, no WebSocket transport. These are protocol limitations rather than model limitations, so you would hit exactly the same wall with any Chat Completions endpoint regardless of what's behind it. For models like GLM, Kimi or DeepSeek V4 Flash driving a normal agentic loop, it is fine. If you need encrypted reasoning, you need a Responses endpoint, full stop.

2. `discover_models`

Codex ships a bundled catalog of OpenAI model metadata. Point it at an Ollama server and `/model` shows you a list of GPT models the endpoint cannot serve, and not the models it can.

The patch adds a `discover_models` flag to `ModelProviderInfo`. When set, the provider's model list is fetched live from its OpenAI-style `/models` endpoint and every reported slug shows up in the picker:

```
[model_providers.fraktal-ollama]
base_url = "http://ml-dev-titan.example.internal:11434/v1"
discover_models = true
```

Push a new model to the Ollama server and it appears in `/model` without anyone editing config. The same flag against OpenRouter surfaces their whole catalog.

Two design decisions worth copying:

Discovered models replace the bundled catalog rather than merging with it. For a provider that reports its own models, that report is authoritative. Merging just repollutes the picker with models the endpoint can't serve.

Discovered models get generic default metadata, not “fallback” metadata. OpenAI-style `/models` only reports slugs, so context window, pricing and capabilities are unknown. There was an existing `model_info_from_slug` path that guesses and flags the result as a fallback, but here the absence of metadata is expected rather than a failure, so a separate `discovered_model_info` keeps the picker clean of spurious warnings.

The flag is local-only and deliberately not plumbed through the remote thread-config surface.

3. Making MCP tools work with local models

This is the patch I’d most like to see upstream, because it’s a host-side bug that only a host can fix.

Codex exposes **MCP** tools to the model as `mcp__<server>__<tool>`, for example `mcp__wren_charts__list_models`. The prefix, the namespace (which is your config key) and the `__` separator are all added host-side; the MCP server itself only ever offers flat names like `list_models`.

Upstream sets `ProviderCapabilities::namespace_tools` to true for every provider, so it emits Responses-API `type: "namespace"` tools unconditionally. Only the OpenAI GPT-5 family implements that convention. Every other model sees the namespace as an opaque wrapper and cannot call the tools inside it, which means **MCP tools are effectively invisible to every non-OpenAI model**.

Flattening namespaced tools into plain top-level functions fixes half of it. The other half is that local models don’t reproduce the wire name exactly:

- the name comes back **flat**, with no separate namespace field, and
- weaker models substitute the `__` separator with `.` or `:`. `qwen3.5:27b` did exactly this.

The result was that upstream’s exact-match lookup returned `unsupported call` for every local model, including `gemma4:26b`, which emitted the perfectly correct `mcp__wren_charts__list_models`, because the flat name was never split back into namespace and tool.

The fix is a tolerant fallback in `core/src/tools/registry.rs` (`resolve_fuzzy_mcp_name / canonical_tool_key`). If the exact lookup misses, normalise the namespace/name boundary by collapsing runs of `.: - _` to a single `_`, then resolve to the one registered MCP tool that matches. **Ambiguous matches are rejected**, so it never guesses, and that constraint is what makes the fallback safe rather than a source of mysterious mis-dispatch. There are unit tests for it in `registry_tests.rs`.

With that, both `qwen3.5:27b` and `gemma4:26b` call MCP tools end to end.

One caveat if you go down this road: MCP Apps servers can expose interactive `ui://` resources ([SEP-1865](#)). A terminal doesn’t render iframes, so in a CLI you get the data plus

a saved PNG path instead of the interactive UI. Tools without a UI render as text as you'd expect.

Fork hygiene

If you are going to maintain a fork of a fast-moving upstream (Codex moved 1,288 commits between two of our rebases) the branch model matters more than the patches.

`main` mirrors upstream untouched, and all fork work lives on `fraktal/main`. In our case that arrangement was forced by an org policy that locks every repo's default branch to `main`, which is annoying, since people landing on the GitHub home page see upstream's README. It turns out to be the right model anyway: a pristine mirror makes `git log upstream/main..fraktal/main` the live, exact list of everything you own.

One commit per intent. Not one commit per file, and not one big "fork changes" commit. Each patch is independently droppable and independently rebasable. When upstream reorganises a module you touched, and they split `codex-mcp/src/connection_manager.rs` into a directory on us, you fix one commit rather than a merge blob.

Gate upstream CI instead of deleting it. We initially deleted `.github/workflows/bazel.yml`, which required remote-cache secrets we don't have. Every single time upstream touched that file we got a delete/modify conflict. Gating it instead:

```
if: github.repository == 'openai/codex'
```

That self-skips on the fork, keeps the scaffolding intact for when we wire up our own credentials, and never conflicts again. Same trick for the release pipeline.

Separate your state directory. Settings, auth, history, sessions and logs live under `~/.fraktal` rather than `~/.codex`, so the fork and a stock Codex install can coexist on the same machine without corrupting each other's state. `CODEX_HOME` is still accepted as a fallback so upstream tooling and scripts keep working unchanged. The repo-local `.codex/` project directory is deliberately left alone, since the fork should still read project config a team has already committed.

Write the rebase runbook before you need it. Ours records what broke on each sync and why. It's the difference between a rebase being a scheduled hour and a rebase being an afternoon of archaeology.

Where this leads

Once the agent can talk to any OpenAI-compatible endpoint, moving between them is a `base_url` change:

```
[model_providers.eu-sovereign]
name = "EU serverless inference"
base_url = "https://api.example-eu-provider.com/v1"
wire_api = "chat"
env_key = "EU_PROVIDER_API_KEY"
```

You keep the same binary, the same agent loop, the same MCP tools and the same muscle memory, and you change the jurisdiction, the processor agreement and the price. That property is what the whole exercise was for, and the [business post](#) covers why it's worth more to a consultancy than any individual model choice.

One of the cosmetic patches turns the agent's own telemetry off by default, which is the right default for a tool handed to consultants. Pointed at a collector you run yourself it becomes useful again, and I've written up [what it takes to get real cost numbers out of it](#) — including the part where Codex's token metrics report zero.

The fork itself is a lightly-patched copy of someone else's very good tool, and most of what it bought us came from removing a coupling rather than from adding a feature. If you end up doing the same thing against a different upstream, I'd be curious what broke on your rebases.